Computer Architecture HW4: Introduction to GPU

Group: A

2026-06-11

Implementation Summary

This submission solves the 2D transient heat equation with fixed Dirichlet boundary conditions. The top boundary is fixed at 100, the left, right, and bottom boundaries are fixed at 0, and the interior is initialized to 0. The corner values on the hot top edge are initialized to 50, the average of the two adjacent boundary values. All runs use

$$r = \alpha \Delta t / h^2 = 0.24$$

which satisfies the 2D explicit stability limit $r \leq 0.25$.

The implementation uses row-major flat arrays. The CUDA versions allocate two device grids, update the interior points, and swap device pointers after each time step. The host does not copy the whole grid during the time loop. Convergence checking uses `thrust::reduce`, as described in Part A.

Platform

Main local platform used for the primary measurements:

Item	Value
CPU	AMD Ryzen 9 7940H, 8 cores / 16 threads
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
CUDA compute capability	8.9
SM count	24
Max threads per block	1024
Shared memory per block	48 KB
Shared memory per SM	100 KB
Approx. peak FP32	13824.00 GFLOP/s
Approx. memory bandwidth	256.03 GB/s
Compiler	`nvcc` 12.0, `-O3 -arch=sm_89`; `g++ -O3`

Pre-lab: Arithmetic Intensity

For one stencil update,

$$T_{i,j}^{n+1} = T_{i,j}^{n} + r(T_{i-1,j}^{n} + T_{i+1,j}^{n} + T_{i,j-1}^{n} + T_{i,j+1}^{n} - 4T_{i,j}^{n})$$

the floating point operation count is:

Operation	Count
Add four neighbors	3 additions
Compute `4*T[i,j]`	1 multiplication
Subtract `4*T[i,j]`	1 subtraction
Multiply by `T[i,j]`	1 multiplication
Add old center value	1 addition
Total	7 FLOPs

Assuming no cache reuse, the memory traffic per update is 5 reads(4 neighbors and T[i,j]) and 1 write(T_new[i,j]) of FP32 values:

$$6 \times 4 = 24 \text{ bytes/update}$$

Thus,

$$I = 7/24 = 0.292 \text{ FLOP/byte.}$$

The GPU ridge point is

$$I_{\text{ridge}} = 13824.00/256.03 = 53.99 \text{ FLOP/byte.}$$

Since $0.292 \ll 53.99$, the stencil is memory-bound on this GPU.

Part 0: Sequential 2D Solver

The sequential solver is in `heat_serial.cpp`. It is a single-threaded CPU reference implementation and is also used to validate the CUDA kernels. For the stationary plot, $N = 256$ converged in 46941 steps with final maximum update 9.918213×10^{-5}.

Baseline CPU timings for the later speedup comparison are:

N	Steps	Time (ms)	Throughput (Gpt/s)
512	2000	980.742	0.530
1024	2000	3843.741	0.543
2048	2000	15809.768	0.530

Part A: Naive CUDA Kernel

The naive CUDA implementation is in `heat_cuda.cu`. Each CUDA thread updates one grid point. For validation, the CUDA result at $N = 256$, 500 steps, and a 16x16 block differed from the CPU reference by 3.814697×10^{-6} in maximum absolute error.

Convergence Reduction

For convergence checking, I used `thrust::reduce` with `thrust::maximum<float>` instead of a hand-written global max-reduction kernel. On checking steps, the CUDA kernel writes each grid point's absolute update difference to a device array `d_diff`. Then `thrust::reduce` computes the global maximum on the GPU. I chose Thrust because it is part of the CUDA toolkit, keeps the reduction code short and less error-prone, and avoids copying the full temperature grid back to the host. The host only receives the final scalar maximum every `check_interval` steps.

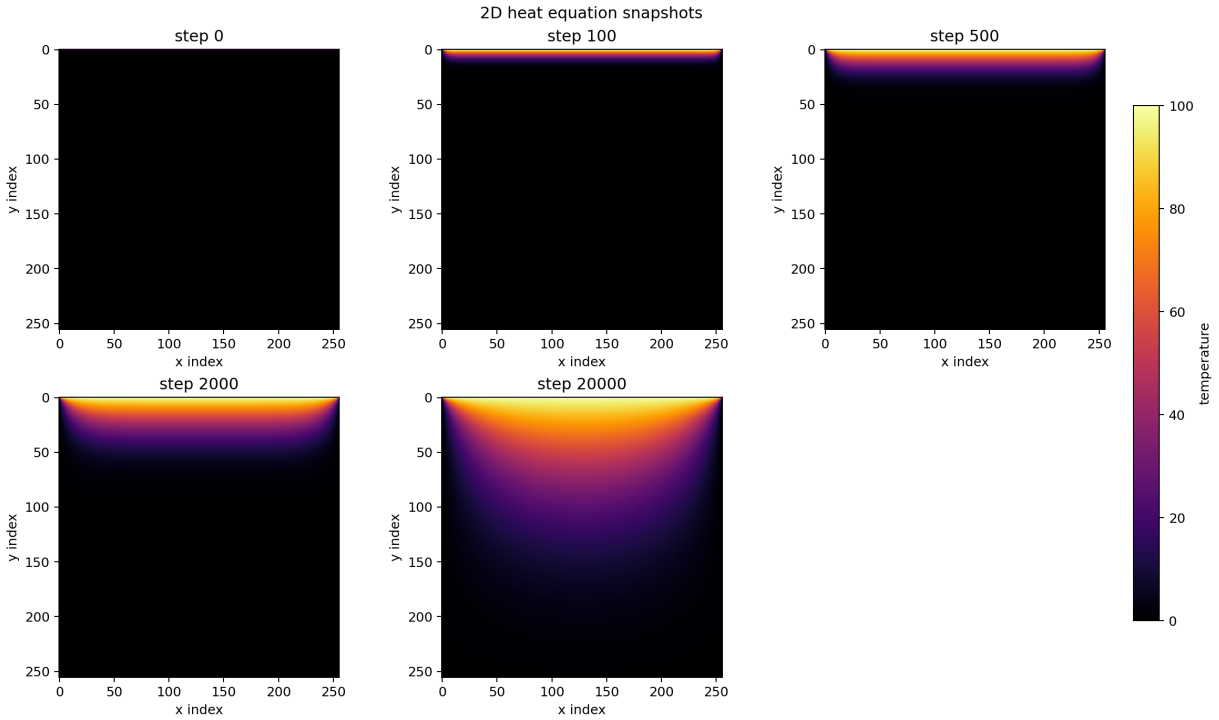

Figure 1: Intermediate CPU snapshots

Block Size Exploration

All runs used $N = 1024$, 2000 steps.

Block size	Threads/block	Time (ms)	Throughput (Gpt/s)
8x8	64	58.312	35.824
16x16	256	62.867	33.228
32x32	1024	59.460	35.132

The best block size in this table is 8x8. Smaller blocks can underperform because they increase block scheduling overhead and may provide less work per resident block. Larger blocks can underperform because a 1024-thread block uses the device limit exactly, leaving fewer resident blocks per SM and less scheduling flexibility for latency hiding.

The maximum number of threads per block on this GPU is 1024. Therefore the 32x32 configuration is legal, since $32 \times 32 = 1024$, but it is exactly at the limit.

Using the best 8x8 result, the effective bandwidth requested in the assignment is:

$$BW_{\text{achieved}} = \frac{(1024-2)^2 \times 24}{58.312 \text{ ms}/2000} = 859.8 \text{ GB/s}.$$

This is an effective stencil bandwidth using the no-cache 24 B/update model. It is higher than the theoretical DRAM bandwidth because the naive kernel benefits from L1/L2 cache reuse between neighboring threads; not all of those 24 bytes are fetched from DRAM every time.

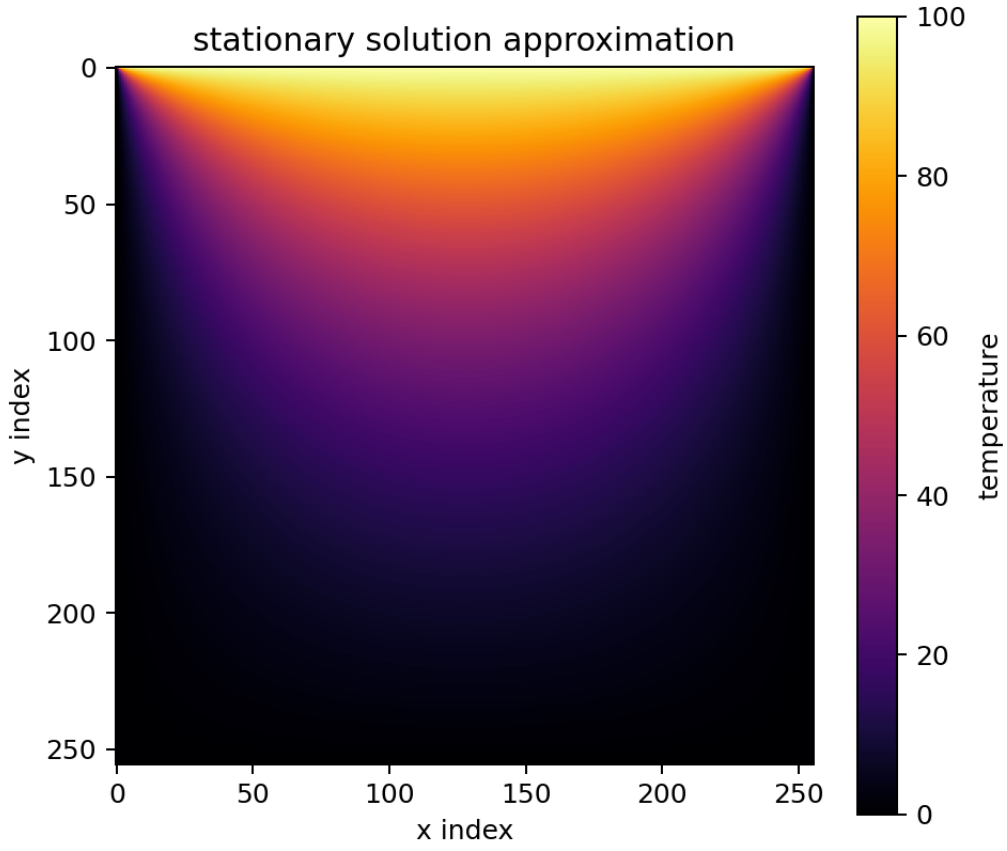

Figure 2: Stationary solution approximation

Speedup Over Serial

This table uses the 8x8 naive configuration selected above.

N	Serial time (ms)	Naive CUDA time (ms)	Speedup
512	980.742	55.376	17.71x
1024	3843.741	58.312	65.92x
2048	15809.768	255.980	61.76x

Part B: Shared-memory Tiled Kernel

The tiled CUDA implementation is in `heat_cuda_tile.cu`. It uses dynamic shared memory with a 1-cell halo around each tile. All stencil reads in the compute phase come from shared memory. Corner halo elements are loaded by corner threads. For validation at $N = 256$, 500 steps, and a 16x16 tile, the maximum absolute error against the CPU reference was 3.814697×10^{-6}.

Halo Loading Compared with MPI Halo Exchange

The shared-memory halo loading in the tiled CUDA kernel is analogous to the MPI halo exchange from the previous assignment, but it happens at a much smaller scope. In MPI, each process exchanges boundary rows or columns with neighboring processes through the network, so the latency is relatively high and depends on message startup cost and interconnect performance. In the CUDA tiled kernel, each thread block loads its own 1-cell halo from global memory into shared memory, so the latency is much lower than network communication, although it still costs global-memory loads and a block-level `__syncthreads()`.

The bandwidth behavior is also different. MPI halo exchange consumes network bandwidth between nodes, while CUDA halo loading consumes GPU global-memory bandwidth and then reuses the loaded values from fast shared memory. This makes the GPU approach cheaper for repeated neighbor access inside a block.

The programming effort is different as well. MPI requires explicit sends, receives, rank-neighbor logic, and synchronization across processes. CUDA halo loading requires careful shared-memory indexing, boundary checks, corner halo loads, and `__syncthreads()`. MPI is more complex at the distributed-system level, while CUDA is more sensitive to low-level indexing and memory-layout details.

Tile Size Exploration

All runs used $N = 1024$, 2000 steps. The speedup column is relative to the best naive result from Part A; values below 1 mean the tiled version was slower on this GPU.

Tile size	Time (ms)	Throughput (Gpt/s)	Speedup vs naive	Shared mem/block (KB)
8x8	105.182	19.861	0.554x	0.391
16x16	90.313	23.130	0.646x	1.266
32x32	127.187	16.424	0.458x	4.516

The GPU provides 100 KB shared memory per SM and 48 KB shared memory per block. The largest stencil tile above uses only 4.516 KB, so none of these tile sizes is limited by shared memory capacity. The convergence reduction is performed by `thrust::reduce` over a separate device difference array, so it does not add extra shared-memory pressure to the tiled stencil kernel.

In principle, shared-memory tiling reduces redundant global loads because a cell loaded into a block's shared tile can be reused by neighboring threads. On this RTX 4060 Laptop GPU, the measured tiled kernel is not consistently faster than the naive kernel. The likely reason is that the naive stencil uses coalesced accesses and

the hardware L1/L2 caches already capture much of the neighbor reuse. The tiled version adds halo-loading branches and a block-wide `__syncthreads()` every time step, which can outweigh the reduced global-memory traffic. For small N, launch overhead and halo overhead are a larger fraction of the total runtime. For large N, cache capacity, memory traffic, and laptop power/clock behavior dominate.

Throughput Versus N

The plot compares the selected naive 8x8 configuration with the best 16x16 shared-memory tile from the tile-size table.

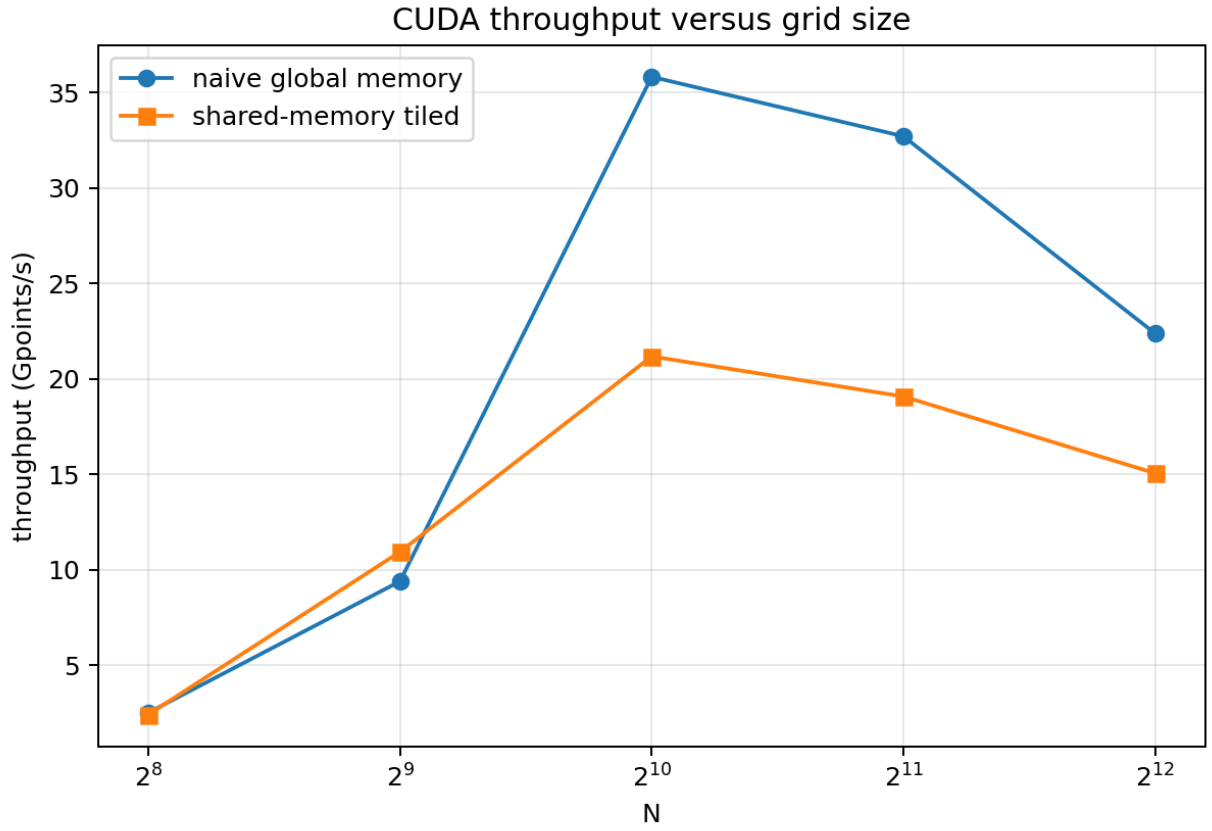

Figure 3: CUDA throughput versus grid size

N	Naive 8x8 (Gpt/s)	Shared 16x16 (Gpt/s)
256	2.511	2.414
512	9.394	10.937
1024	35.824	21.178
2048	32.707	19.077
4096	22.381	15.058

Both selected configurations peak at $N = 1024$ in this experiment. For small grids the GPU is underutilized and kernel launch overhead is significant. As the grid grows, occupancy and memory-level parallelism improve. At the largest sizes, the working set no longer benefits as much from cache locality and the longer laptop-GPU runs are more sensitive to memory bandwidth and power limits.

Notes on Reduction Approach

The convergence-reduction method is described in Part A. The same `thrust::reduce` approach is reused for the shared-memory kernel in Part B, so both CUDA versions check convergence without copying the full grid back to the host.

Files

File	Purpose
`heat_common.hpp`	Shared initialization, CPU reference solver, CSV output, result formatting
`heat_serial.cpp`	Sequential 2D solver
`heat_cuda.cu`	Naive CUDA global-memory solver
`heat_cuda_tile.cu`	Shared-memory tiled CUDA solver
`cuda_common.cuh`	CUDA error checking and device-spec helpers
`cuda_device_info.cu`	Platform information tool
`Makefile`	Build rules
`scripts/run_benchmarks.sh`	Reproduces validation, benchmark data, and plots
`scripts/plot_results.py`	Generates plots and parsed CSV

Encountered Problems and Solutions

The main performance surprise was that the shared-memory kernel did not outperform the naive kernel for the larger runs. This was checked against the CPU reference, so the issue was not a numerical error. The explanation is that maybe the naive kernel is highly cache-friendly on this GPU, which means that caches help reducing the time to access those memories, while the tiled kernel pays extra synchronization and halo-loading overhead. I therefore report the measured behavior directly and describe the hardware-cache effect.

Another implementation detail was avoiding per-step host transfers. The solution was to run a reduction only on selected checking steps and copy back only one float per block.